Module 6: C# Other New features

GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

Implicit Typed Variables

- Explicitly typed variable

```
int i = 10; //explicitly typed
```

- Implicit type **var**

```
var i = 10; // implicitly typed
```

- Strongly typed (Compiler determines the type)
- Value must be assigned
- >= C# 3.0

Using Implicit typed variables

- Declaring variables

```
System.Text.StringBuilder sb
        = new System.Text.StringBuilder();
```

```
var sb = new System.Text.StringBuilder();
```

- Using foreach loop

```
foreach (KeyValuePair<string, string> entry in myDictionary)
{}
```

```
foreach (var entry in myDictionary)
{}
```

Restrictions of var

- Value must be assigned

```
var i;
```

- Can not be assigned null

```
var i = null;
```

- Can't agregate implicit variable declaration

```
var i = 1, j = 2;
```

- Can be used only for local variables, not class members, not method parameters, not method return values

```
class Employee
{
        public var Size = 1;
}
```

Overview

- Implicit Typed Variables
- **Constructor Invocation**
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

Constructor Invocation in C# 9.0

- If a target type of an expression is known, you can omit a type name

```
StringBuilder sb = new StringBuilder();
```

```
var sb = new StringBuilder();
```

SYNTACTIC SUGAR

- or

```
StringBuilder sb = new ();
```

SYNTACTIC SUGAR

Overview

- Implicit Typed Variables
- Constructor Invocation
- **Partial Classes and Methods**
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

Partial Classes

- Possibility of splitting the definition of a class or a struct, an interface or a method over two or more source files

```
public partial class Employee
{
        public string FirstName { get; set; }
}
public partial class Employee
{
        public string LastName { get; set; }
}
```

- All the parts must use the **partial** keyword
- >= C# 2.0

Restrictions of Partial Classes

- All the parts must use the **partial** keyword
- All the parts must have the same accessibility, such as **public**, **private**, and so on
- Must be declared in same **Assembly**, **Module** and **Namespace**

Note:

- *Generic types can also be partial.*
  (Each partial declaration must use the same parameter names in the same order)

Partial Methods

- Method that may or may not be implemented

```
partial class  MyClass
{
    partial void SayHello();

    public void MakeGreeting()
    {
        SayHello();
        Console.WriteLine("World");
    }
}

partial class  MyClass
{
    partial void SayHello()
    {
        Console.WriteLine("Hello");
    }
}
```

```
internal class MyClass
{
    private void SayHello()
    {
        Console.WriteLine("Hello");
    }

    public void MakeGreeting()
    {
        SayHello();
        Console.WriteLine("World");
    }
}
```

is implemented

GOPAS

Partial Methods

- Method that may or may not be implemented

```
partial class  MyClass
{
    partial void SayHello();

    public void MakeGreeting()
    {
        SayHello();
        Console.WriteLine("World");
    }
}

partial class  MyClass
{
}
```

```
internal class MyClass
{
    public void MakeGreeting()
    {
        Console.WriteLine("World");
    }
}
```

is not implemented

GOPAS

Partial Private Methods

- If the implementation is not supplied, then the method and all calls to the method are removed at compile time

- Restrictions:
  - *must be **void***
  - *not out parameters*
  - *and only one implementation part is allowed*

Tip:

- *Useful as a way to customize generated code (developer can decide whether to implement the method)*

Partial Not Private Methods

- Implementation must be provided
- Does not have to be **void**

```
partial class  MyClass
{
    public partial string GetHello();

    public void MakeGreeting()
    {
        SayHello();
    }
}

partial class  MyClass
{
    public partial string GetHello() // has to be provided
    {
        return "Hello";
    }
}
```

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- **Optional and Named Parameters**
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

Optional Parameters

- Optional parameter has a default value

```
int Calculate(int requiredParameter, int optionalParameter = 0)
```

- Can be used in declaration of
  - method
  - constructor
  - indexer
  - delegate

>=.Net 4.0

Named Parameters

- Names of parameters

```
int Calculate( int a = 0, int b = 0)
```

- can be called as

```
Calculate(10, 20);
```

```
Calculate(10);
```

- or

```
Calculate(a:10, b:20)
```

- or

```
Calculate(b:20, a:10)
```

- or

```
Calculate(b:20)
```

Restrictions of Optional and named arguments

- Optional parameters must be after required parameters

```
int Calculate(int a, int b = 0, int c = 0)
```

- Named argument specifications must appear after all fixed arguments have been specified

```
Calculate(10, c:20)
```

Note:

- *In case of overloading preference goes to a candidate that does not have optional parameters*

Lab 06.1

Exercise:

1. Implicitly Typed Variables
2. Constructor Invocation
3. Partial
   - Class
   - Interface
   - Method
4. Optional Parameters and Overloading

GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- **Nullable Value/Reference Types**
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

Nullable Value Types

- Nullable value types represent value-type variables that can be assigned the value of **null**
- Nullable types are instances of the **System.Nullable<T>** struct
- >= C# 2.0

Note:

- *cannot create a nullable type based on a reference type. (Reference types already support the **null** value.)*

Nullable Value Types

- System.Nullable<T> struct

```
System.Nullable<int> i = null;
```

- shorthand for Nullable<T>

```
int? i = null;
```

Nullable types and property Value

- Value - Gets the value if there is any, otherwise InvalidOperationException will be thrown

```
int? a = 1;
int? b = null;

int? j = a; // OK
int? k = b; // OK

// Cannot implicitly convert nullable to int
int l = a; // Syntax Error

int m = a.Value; // OK
int n = b.Value; // InvalidOperationException
                 // Nullable object must have a value.
```

Nullable types and property HasValue

```
int? b = null;
if (b.HasValue)
{
        int o = b.Value; // OK
}
```

or

```
int? b = null;
if (b != null)
{
        int o = b.Value; // OK
}
```

?? null-coalescing operator >= *C# 2.0*

- GetValueOrDefault() - Gets value or default value (0/false)

```
int? x = null;
int y = x.GetValueOrDefault();
```

or

```
int? x = null;
int y = x ?? 0;
```

- operator ?? returns the left-hand operand if the operand is not null, otherwise it returns the right hand operand.

??= null-coalescing assignment operator *>= C# 8.0*

- assigns the value of its right-hand operand to its left-hand operand only if the left-hand operand evaluates to null

```
if (list == null)
{
    list = new List<int>();
}
```

```
list ??= new List<int>();
```

- Example:

```
string s1 = null;
s1 ??= "Hello";
Console.WriteLine(s1);  // Hello
s1 ??= "World";
Console.WriteLine(s1);  // Hello, since the s1 is not null
```

?. or ?[Null-conditional operator >=C# 6.0

- Applies operation to its operand only if that operand evaluates to non-null; otherwise, it returns null

```
int? length;
if (customers == null)
        length = null;
 else
        length = customers.Length;
```

or

```
int? length = (customers==null)? null : customers.Length;
```

or

```
int? length = customers?.Length; // null if customers is null
```

Null-condition member access

- *Operator* ?.

```
int? length = customers?.Length; // null if customers is null
```

- *Operator* ?[

```
Customer first = customers?[0];  // null if customers is null
```

- Combination operators

```
int? count = customers?[0]?.Orders?.Count;  // null if customers,
the first customer, or Orders is null
```

or

```
int count = customers?[0]?.Orders?.Count ?? 0;  // zero if
customers, the first customer, or Orders is null
```

Note:

- *Still IndexOutOfRange if item does not exists*

Invoking methods

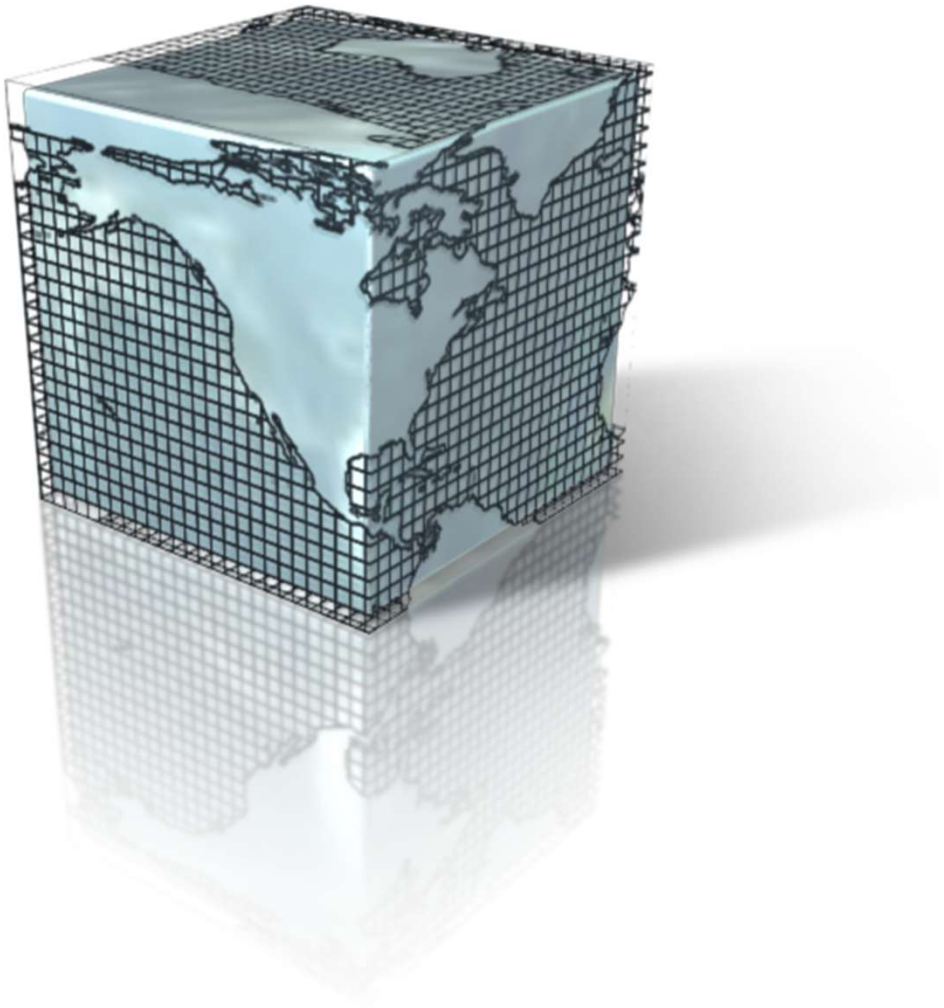
- Invoking delegates

```
if (handler != null) handler.Invoke(e);
```

or

```
handler?.Invoke(e);
```

Lab 06.2 - Nullable Value Types



Exercise:

1. Nullable Value Types
2. Operator ??
3. Operator ?. and ?[
4. Operator ??=

GOPAS

Nullable Reference Types >=C# 8

Beware of NullReferenceException

```
string s = Console.ReadLine();
Console.WriteLine(s.Length); // The Pitfall of Null
```

If the Ctrl+Z method returns null

- Tip: Enable reference nullable types

```
string s = Console.ReadLine();  // Warning: Possible Null
Console.WriteLine(s.Length);    // Warning: Possible Null
```

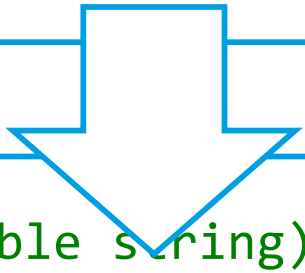
- Designed to help better handle situations where the value might be null to prevent errors

GOPAS

Preventing Nullable Warnings

- Proper Null Checks for Safety

```
string? s = Console.ReadLine();       // OK (nullable string)
Console.WriteLine(s.Length);          // Warning: Possible Null
```



```
string? s = Console.ReadLine();       // OK (nullable string)
if (s != null)
{
    Console.WriteLine(s.Length);      // OK, because it's checked
}
```

Smart way of preventing Nullable Warnings

a) Using condition

```
if (s != null)
{
    Console.WriteLine(s.Length); // OK, because it's checked
}
```

b) Use null-conditional operator

```
Console.WriteLine(s?.Length); // OK, return null if is null
```

c) Use null-coalesting operator

```
Console.WriteLine(s?.Length ?? 0); // OK, return 0 if null
```

GOPAS

Handling Nullable Reference Type Warnings

- Nullable warnings enabled by default

```
string? s = null;                    // OK (nullable string)
Console.WriteLine(s.Length);         // Warning: Possible Null
```

- Disable warning using compilator directive

```
string? s = null;                    // OK (nullable string)
#nullable disable
Console.WriteLine(s.Length);         // OK, Warnings disabled
```

- Disable warning using null-forgiving operator

```
string? s = null;                    // OK (nullable string)
Console.WriteLine(s!.Length);        // OK, Warnings disabled
```

Beware null-forgiving operator

a) Use null-conditional operator

```
Console.WriteLine(s?.Length);          // OK (solved)
```

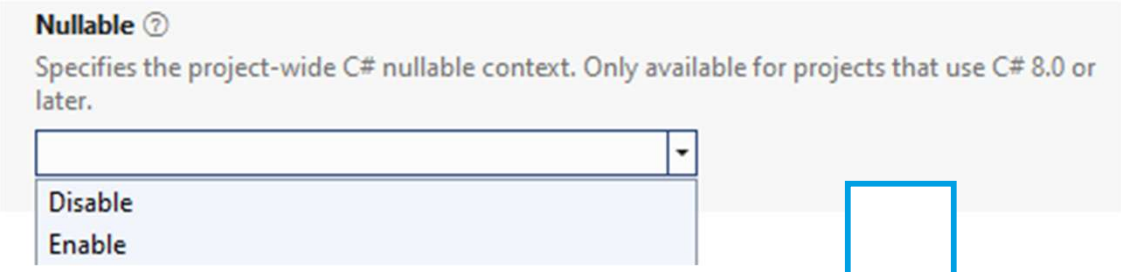
b) Use null-forgiving operator

```
Console.WriteLine(s!.Length);          // OK (I don't care)
```

- Ignore the possibility that the variable could be null
- Removes any Nullable warnings
- Shifts the responsibility to you

Enabling at Project Level

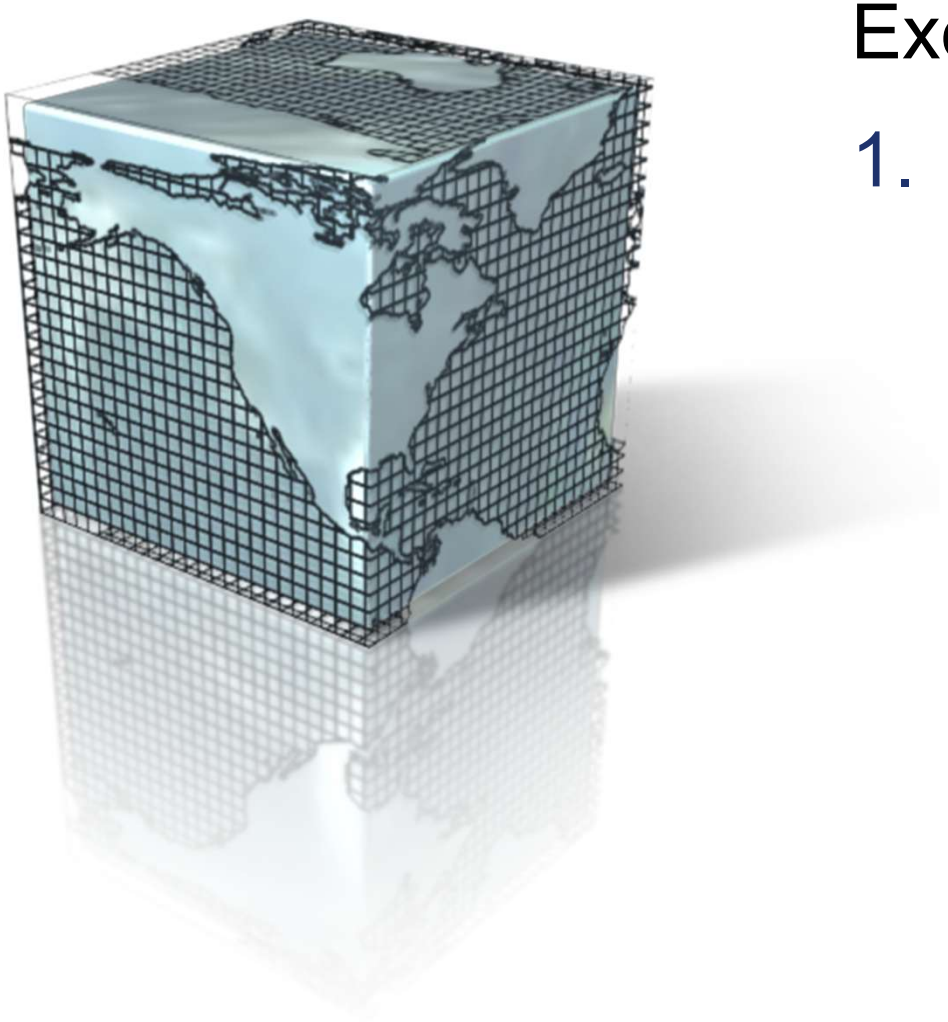
- Enabled by default from .NET6
- Project Properties in VS2022




```
<PropertyGroup>
        <OutputType>Exe</OutputType>
        <TargetFramework>net6.0</TargetFramework>
        <LangVersion>8.0</LangVersion>
        <Nullable>enable</Nullable>
</PropertyGroup>
```

GOPAS

Lab 06.3 - Nullable Reference Types



Exercise:

1. Nullable Reference Types

GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- **Tuples**
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

GOPAS

System.Tuple<> generic class

- Reference type
- Simple class that can contain more elements (2 to 8)
- Generic class => Strongly typed

```
Tuple<int, string, bool> values
        = new Tuple<int, string, bool>(10, "Joe", true);

Console.WriteLine(values.Item1); // 10
Console.WriteLine(values.Item2); // "Joe"
Console.WriteLine(values.Item3); // true
```

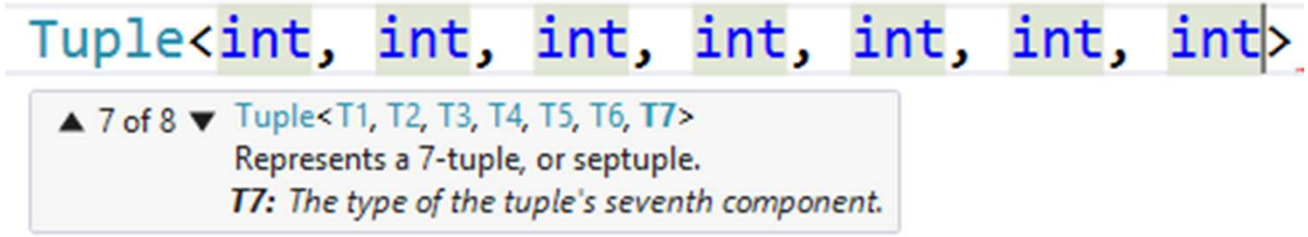
- Is read only

```
values.Item3 = false; // read only
```

GOPAS

Restrictions of Tuples

- 8 overloads "only"



Tip:

- *You can create Tuple of Tuple*

System.ValueTuple<> generic struct >=C# 7.0

- value type can be more effective (reference type is allocated on heap too many objects => too many object creation/allocation)

a) Tuple as a reference type

```
Tuple<int, string, bool> values
        = new Tuple<int, string, bool>(10, "Joe", true);
```

b) Tuple as a Value type (>=C# 7.0)

```
ValueTuple<int, string, bool> values
        = new ValueTuple<int, string, bool>(10, "Joe", true);
```

Read / Write access

a) Reference Tuple is Read-Only

```
Tuple<int, string, bool> values
        = new Tuple<int, string, bool>(10, "Joe", true);
values.Item1 = 1; // Error: Is read Only
```

b) Value Tuple is Read / Write

```
ValueTuple<int, string, bool> values
        = new ValueTuple<int, string, bool>(10, "Joe", true);
values.Item1 = 1; // OK: Is Read / Write
```

System.ValueTuple<> generic struct >=C# 7.0

```
ValueTuple<int, string, bool> values
        = new ValueTuple<int, string, bool>(10, "Joe", true);
```

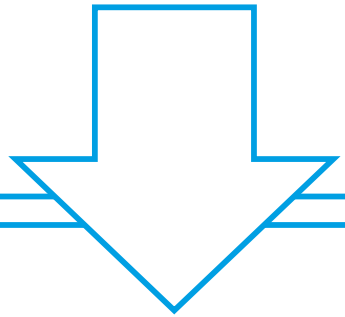
- It can be created and initialized using parentheses ()

```
(int, string, bool) values = (10, "Joe", true);
```

Naming Value type Tuple's properties

- Default names for tuple elements: Item1, Item2, Item3..
- You can specify more descriptive names

```
(int, string, bool) getResult()
{
    return (10, "Joe", true);
}
```



```
(int, string, bool) result = getResult();
Console.WriteLine(result.Item1);
Console.WriteLine(result.Item2);
Console.WriteLine(result.Item3);
```

- or

```
(int Id, string Name, bool IsMember) result = getResult();
Console.WriteLine(result.Id);
Console.WriteLine(result.Name);
Console.WriteLine(result.IsMember);
```

Naming Value type Tuple's

```
(int Id, string Name, bool IsMember) getResult()
{
    return (10, "Joe", true);
}
```

```
(int Id, string Name, bool IsMember) result = getResult();
Console.WriteLine(result.Id);
Console.WriteLine(result.Name);
Console.WriteLine(result.IsMember);
```

- or

```
var result = getResult();
Console.WriteLine(result.Id);
Console.WriteLine(result.Name);
Console.WriteLine(result.IsMember);
```

Inferred tuple element names

- >=C# 7.0

```
int id = 1;
string name = "Joe";
bool isMember = true;

var customer = (Id:id, Name:name, IsMember:isMember);
```

- >=C# 7.1

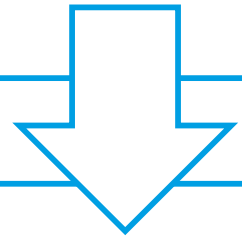
```
int Id = 1;
string Name = "Joe";
bool IsMember = true;

var customer = (Id, Name, IsMember);
```

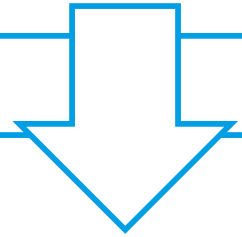
Tuple Naming

- From Default Items to Explicit Property Names

```
int number = 1;
var calculation = (number, number * 10);
Console.WriteLine($"{calc.number}: {calc.Item2}");
```

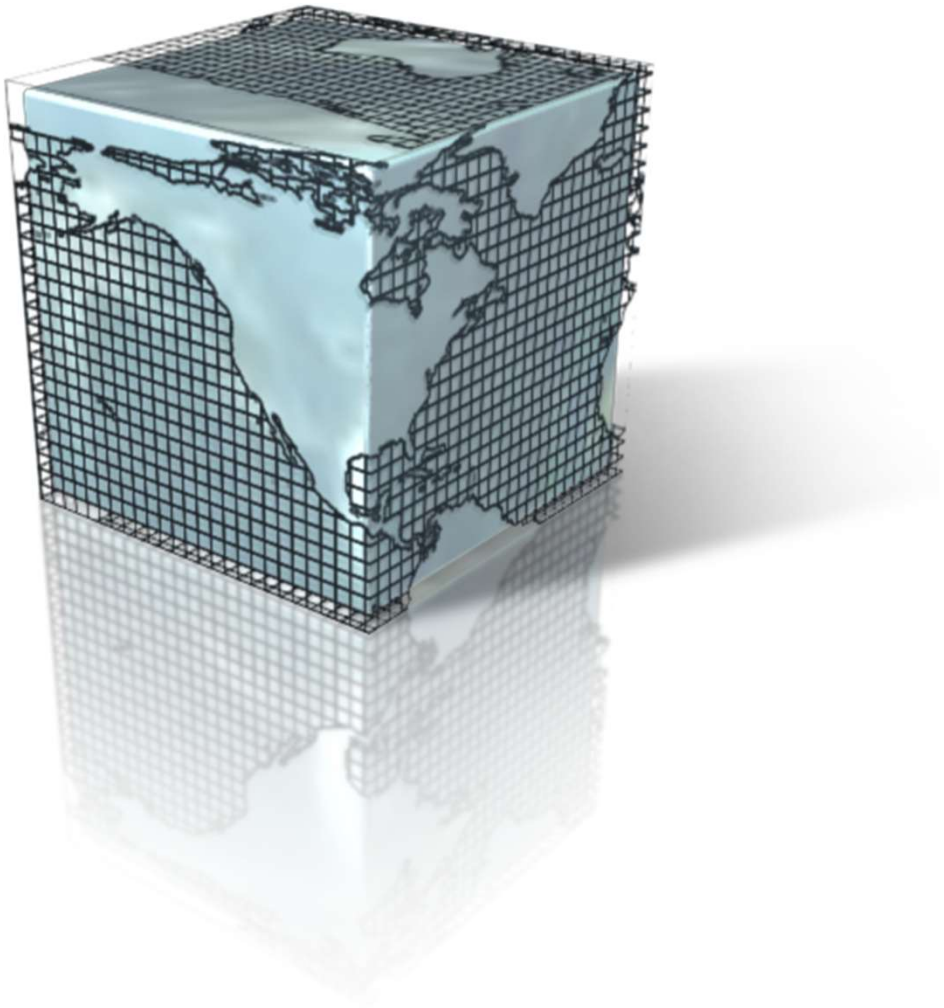


```
int number = 1;
var calc = (number, Product: number * 10);
Console.WriteLine($"{calc.number}: {calc.Product}");
```



```
int number = 1;
var calc = (Number: number, Product: number * 10);
Console.WriteLine($"{calc.Number}: {calc.Product}");
```

Lab 06.4 - Tuples



Exercise:

- Reference type Tuple
- Value type Tuple

GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- **Out variables, Discards and Deconstructions**
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

GOPAS

Out Variables >= C#7

- You can define a method's out parameters directly in the method

```
int result;
if (int.TryParse(input, out result))
{
    Console.WriteLine("Number: " + result);
}
```

```
if (int.TryParse(input, out int result))
{
    Console.WriteLine("Number: " + result);
}
```

SYNTACTIC SUGAR

```
if (int.TryParse(input, out var result))
{
    Console.WriteLine("Number: " + result);
}
```

SYNTACTIC SUGAR

Discards variable >= C#7

- Discards are local variables which you can assign a value to them and that **value cannot be read** (is discarded).
- In essence they are “write-only” variables.
- They do not have names, but rather they are represented with a _ (underscore).

```
_ = 1;                          // OK
Console.WriteLine(_);    // Compilation error
//The name '_' does not exist in the current context
```

GOPAS

Discards variable >= C#7

- Used when you **have to** provide a variable, but don't need it.

```
if (!int.TryParse(input, out _))
{
    Console.WriteLine("Not a number");
}
```

Deconstructions of Tuples >= C#7

- Built-in support for deconstructing tuples

```
var tuple1 = (10, 20, 30);
```

- Unpackage all the items in a tuple in a single operation

```
int a, b, c;
(a, b, c) = tuple1;
```

or

```
(int a, int b, int c) = tuple1;
```

or

```
var (a, b, c) = tuple1;
```

or

```
var (_, _, c) = tuple1;
```

GOPAS

Deconstructing user-defined types

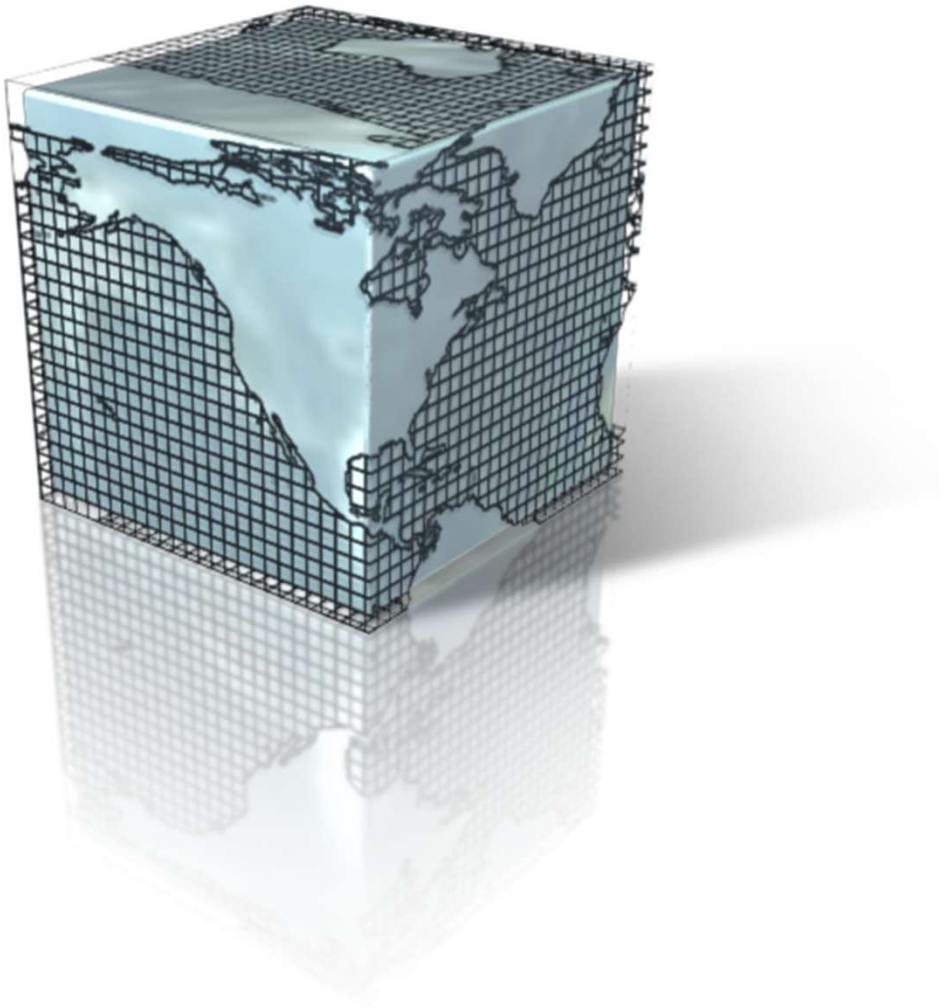
- Built-in support only for deconstructing tuples
- Custom deconstruction for user-defined types
  - Returns void
  - Each value to be deconstructed is indicated by an out parameter

```
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }

    public void Deconstruct(out int id, out string name)
    {
        id = this.Id;
        name = this.Name;
    }
}
```

```
Employee employee = new Employee();
var (id, name) = employee;
```

Lab 06.5 – Discard operator and deconstruction



Exercise:

- Discard operator
- Deconstruction of Tuple
- Deconstruction of User-Defined Type

GOPAS

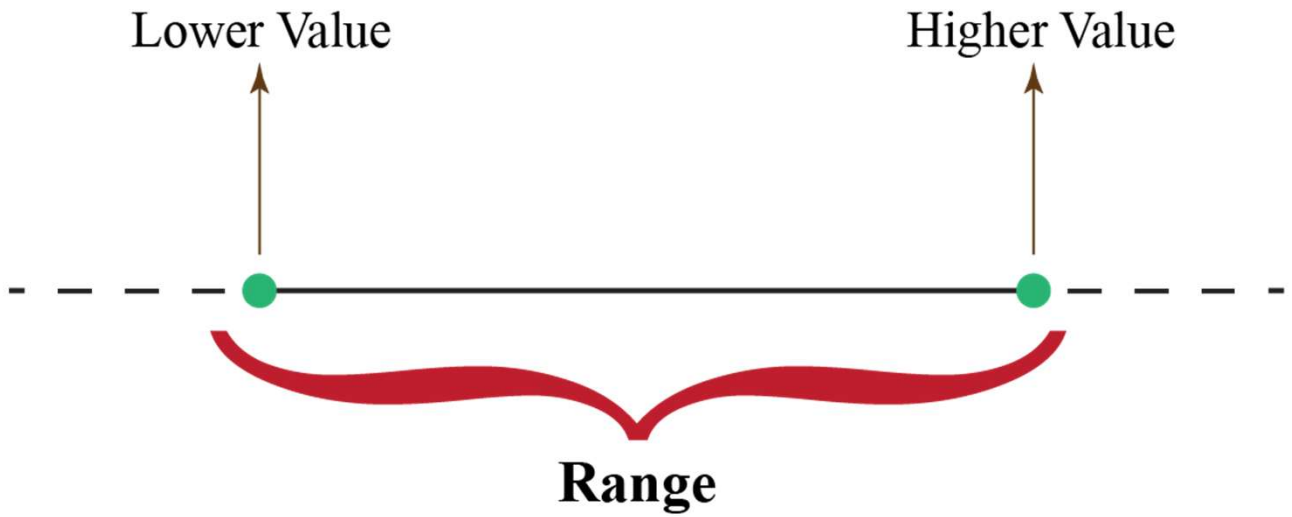
Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- **Index and Range Operator**
- Extension Methods
- Object Initializers and Init Only Setters

C# Index and Range Operators (C# >=8.0)

= C#8 added operators:

- ^ = index operator
- .. = range operator

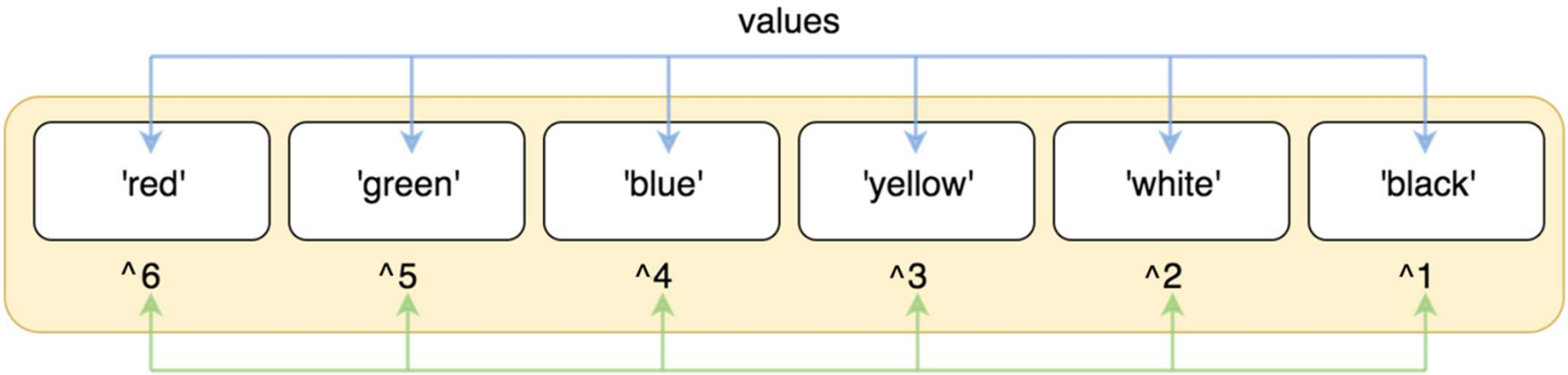


GOPAS

Index operator (C# >=8.0)

- Indexing elements from the last element

```
string[] colors = { "red", "green", "blue", "yellow", "white", "black" };
```




```
Console.WriteLine(colors[^1]);  // black
Console.WriteLine(colors[^2]);  // white
Console.WriteLine(colors[^3]);  // yellow
colors[^1] = "pink"; //change last item value to "pink"
```

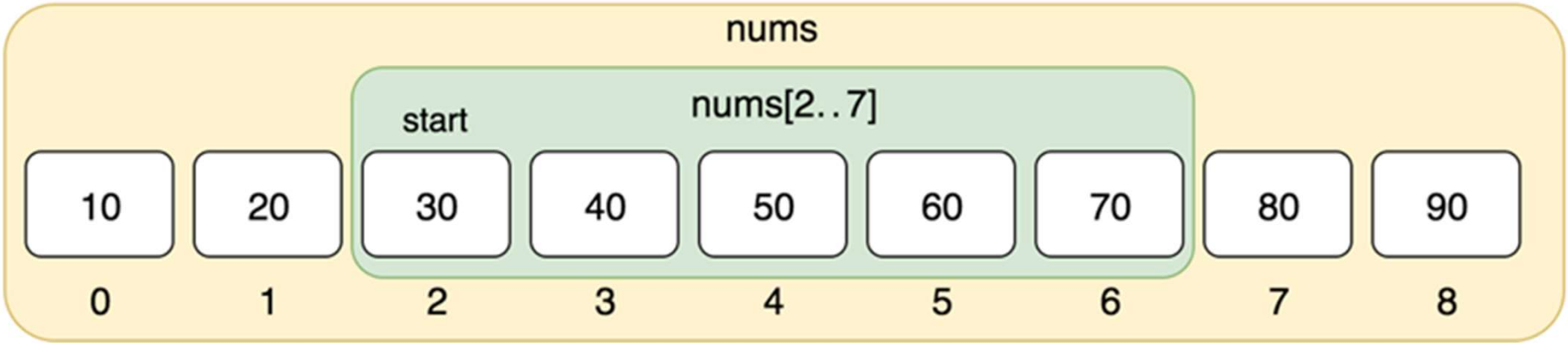
GOPAS

Range operator

(C# >=8.0)

- Gets a slice from collection

```
int[] nums = { 10, 20, 30, 40, 50, 60, 70, 80, 90 };
```




```
Console.WriteLine(nums[2..7][1]); // 40
```

GOPAS

Inclusive Start and Exclusive End

- Range expression that selects all elements

```
int[] nums = { 10, 20, 30, 40 };
nums.Length                 // length: 4
nums[0..nums.Length]        // range: 0..4
```

  - start index 0 (inclusive)
  - end index 4 (exclusive)

Note:

- *[start..end] indicates a range with an **inclusive start** and an **exclusive end***

Using Ranges

- Positive Indexing

```
int[] nums = { 10, 20, 30, 40, 50, 60, 70, 80, 90 };
nums[0..4]          // {10, 20, 30, 40}
nums[..4]// {10, 20, 30, 40}
nums[..] // {10, 20, 30, 40, 50, 60, 70, 80, 90 }       Tip: shallow copy
```

- Negative Indexing

```
nums[^1]            // {90}
nums[^2..^1]        // {80}
nums[^2..^0]        // {80, 90}
nums[^2..]          // {80, 90}
nums[..^1]          // {10, 20, 30, 40, 50, 60, 70, 80}
nums[2..^1]         // {30, 40, 50, 60, 70, 80}
```

- Using variable

```
int fromLast = 1;
nums[..^fromLast]  // {10, 20, 30, 40, 50, 60, 70, 80}
```

- Assigning to a slice is not possible

```
nums[0..4] = nums[1..2];    // not possible
```

Lab 06.06

Exercise:

- Index operator
- Range operator

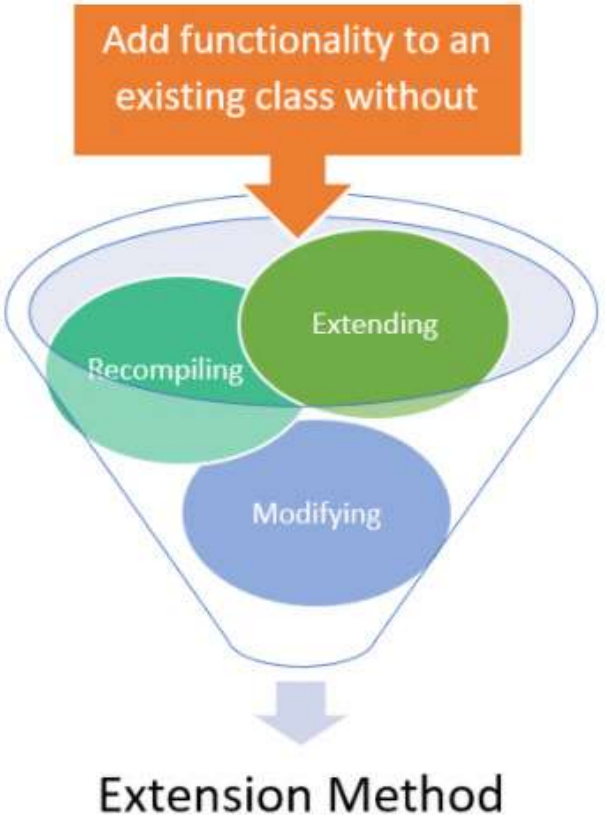
GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- **Extension Methods**
- Object Initializers and Init Only Setters

Extension Methods >= *C# 3.0*

- Enable you to "add" methods to existing types
  (No need to create a new derived type, recompile or modifying the original type)



Tip: Very useful in extending sealed classes

Tip: Extend the type by extending the interface

GOPAS

From Helper Class to Extension Method

- Helper Class

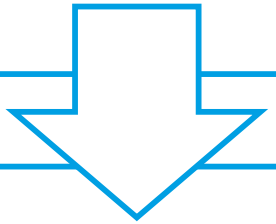
```
static class StringExtensions
{
    public static int GetLength(string that)
    {
        return that.Length;
    }
}

int length = StringExtensions.GetLength("");
```

- Extension method

```
static class StringExtensions
{
    public static int GetLength(this string that)
    {
        return that.Length;
    }
}

int length = "".GetLength();
```



Extension Methods

- Must be located in a **static class**
- Created like **static method**
- Defined as static methods but called like instance methods

```
static class StringExtensions
{
    public static int GetLength(this string that)
    {
        return that.Length;
    }
}
```

```
int length = "".GetLength();
```

Overloading Extension Methods

- Extension Method can be overloaded

```
static class StringExtensions
{
    public static int CharAt(this string that, int index)
    {
        return that[index];
    }
}
Console.WriteLine("ABC".CharAt(1));          // 'B'
```

Extension Methods

- Are only in scope when you explicitly import the namespace with a using directive

```
using MyStringExtensions;
```

- Can extend both
  - value types
  - reference types / interfaces

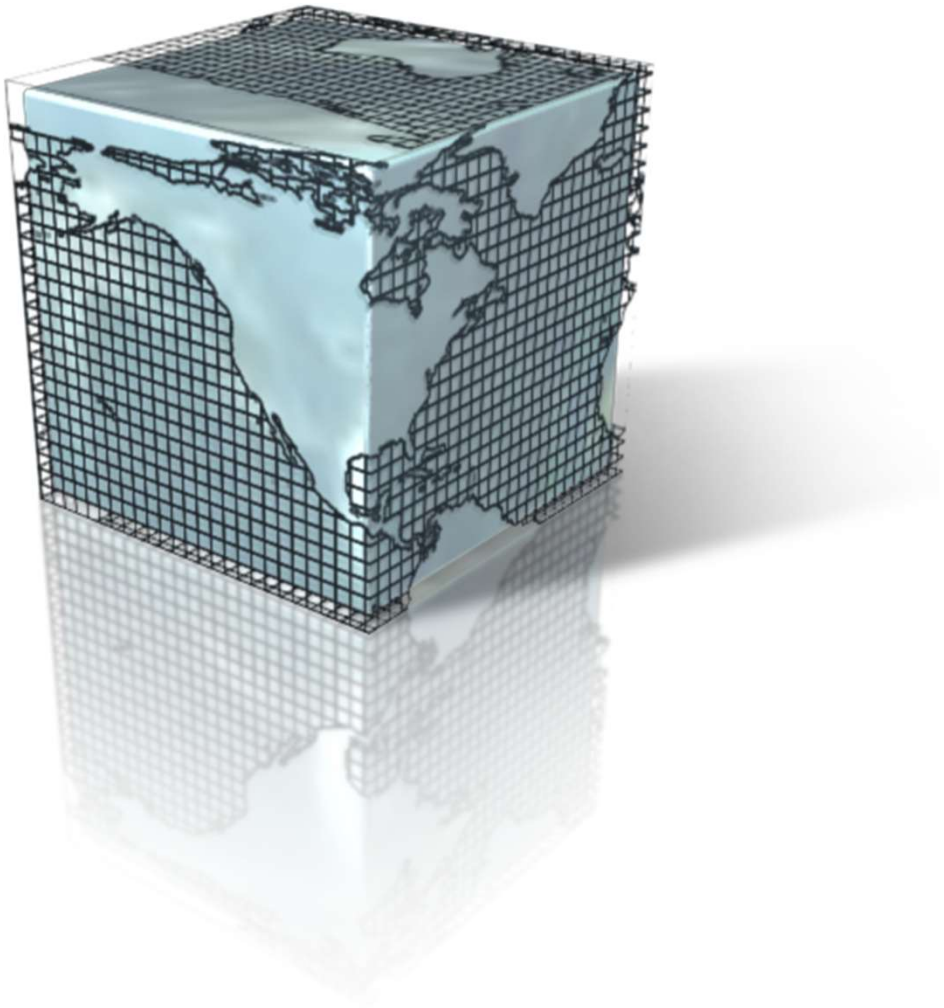
- Restrictions:
  - Must be top level Class (can not be nested)
  - Cannot use private data (it does not belong to the object)
  - Cannot be used to override existing methods
  - Cannot be applied to fields, properties or events

Note:

- *Extension method can be used even there is null reference*

GOPAS

Lab 06.7 - Extension methods



Exercise:

1. Extension methods
2. Overloading
3. Private members

GOPAS

Overview

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- **Object Initializers and Init Only Setters**

Object and Collection Initializers

- Property initialization

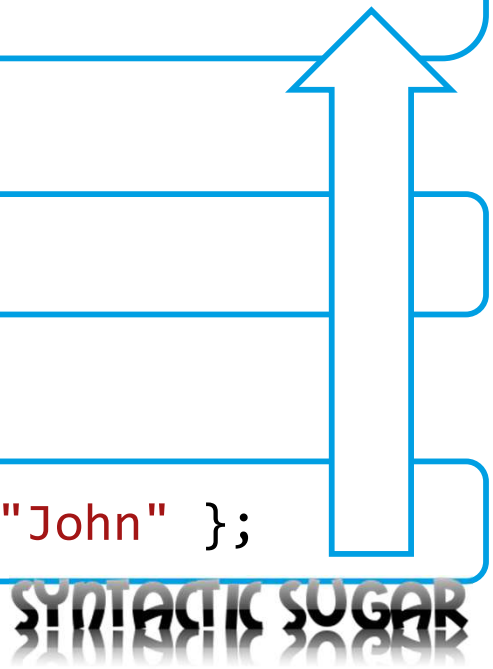
```
Employee e1 = new Employee();
e1.Id = 1;
e1.Name = "John";
```

- Constructor initialization

```
Employee e1 = new Employee(1, "John");
```

- Objects initializer

```
Employee e1 = new Employee(){ Id = 1, Name = "John" };
```



Collection Initializers

- Initializing an Array

```
int[] list = new int[3] { 10, 20, 30 };
```

- To initialize a Collection

```
List<int> digits = new List<int>() { 0 + 1, 12 % 3, MakeInt() };
```


```
List<int> list2 = new List<int>();
list2.Add(0 + 1);
list2.Add(12 % 3);
list2.Add(MakeInt());
List<int> list = list2;
```

Note:

- The compiler adds the calls to .**Add()** method
- For Custom Collection you must create method .Add()
- .Add() can be also extension method

GOPAS

Index initializers >= C# 6.0

- You can initialize a dictionary with collection initializers

```
Dictionary<string, string> list = new Dictionary<string, string>()
{
        ["CZ"] = "Czech Republic",
        ["FR"] = "France"
};
```

- Can be used for any object supporting an indexer

```
List<string> list = new List<string>(new string[] { "A", "B", "C", "D" })
{
        [1] = "b",
        [3] = "d"
};
```

Private setters

- Private setter is accessible only from class itself

```
public class Employee
{
    public string FirstName { get; private set; }
    public string LastName { get; private set; }
}
```

```
Employee employee1 = new Employee()
{
        FirstName = "John",    // Error (private setter)
        LastName = "Doe"       // Error (private setter)
};
```

GOPAS

Init only setters and Object Initializer

- Accessible from Object Initializer

```
public class Employee
{
    public string FirstName { get; init; }
    public string LastName { get; init; }
}
```

```
Employee employee1 = new Employee()
{
        FirstName = "John",    // OK (initialization)
        LastName = "Doe"       // OK (initialization)
};
Employee1.FirstName = "Paul" // ERROR (not initialization)
```

GOPAS

Init only setters >= C# 9.0

- Accessible from
  - Constructors
  - Object Initializers
  - Property Initializers
  - With-expressions
- Readonly once construction has completed

```
public string FirstName { get; init; }
```

GOPAS

Init only setters and Constructor

- Accessible from Object Constructor

```
public class Employee
{
    public string FirstName { get; init; }
    public string LastName { get; init; }

    public Employee(string firstName, string lastName)
    {
        this.FirstName = firstName;
        this.LastName = lastName;
    }
};
```

```
Employee employee1 = new Employee("John", "Doe");  // OK
Employee1.FirstName = "Paul" // ERROR (not initialization)
```

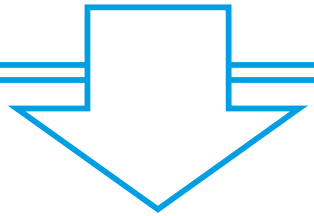
GOPAS

Init only setters and Property Initializers

- Accessible from Property Initializers

```
public class Employee
{
    public string FirstName { get; init; } = "";
    public string LastName { get; init; } = "";
};
```

```
public class Employee
{
    public string FirstName { get; init; } = default(string);
    public string LastName { get; init; } = default(string);
};
```



```
public class Employee
{
    public string FirstName { get; init; } = default;
    public string LastName { get; init; } = default;
};
```

Init only setters in records

- Accessible from With-expressions using records

```
Employee employee1 = new Employee("John", "Doe");
Employee employee2 = employee1 with { LastName = "Parker" };
Console.WriteLine(employee2);          // John Parker
```

- Can be used for:
  a) record type (C# 9.0)
  b) struct (C# 10.0)
  c) annonymous type (C# 10.0)

Lab 06.8 - Object Initializers and Init-only Properties

Exercise:

1. Object Initializers
2. Collection Initializers
3. Extension method .Add()
4. Index Initializers (C# 6.0)
5. Init-only Setters (C# 9.0)

GOPAS

Exercise - Tuples + Extension method + Object initializers

```
class Employee
- FirstName
- LastName
- Age
```

```
// 1) Use object initializer to init:
//      FirstName = "John"
//      LastName = "Doe"
//      Age = 30


// 2) Do not allow to change value after initialization
employee.FirstName = "Paul"; // Fails to compile

// 3) Use Tuple to return more values from GetEmployeeDetails()
var (fullName, initials) = employee.GetEmployeeDetails();

// 4) Use Extension Metody to format output
Console.WriteLine(employee.FormatEmployeeInfo());
Console.WriteLine($"Full Name: {fullName}, Initials: {initials}");
```

Expected output:

```
Employee: John Doe, Age: 30
Full Name: John Doe, Initials: J.D.
```

Review

- Implicit Typed Variables
- Constructor Invocation
- Partial Classes and Methods
- Optional and Named Parameters
- Nullable Value/Reference Types
- Tuples
- Out variables, Discards and Deconstructions
- Index and Range Operator
- Extension Methods
- Object Initializers and Init Only Setters

GOPAS